

03-optimization-techniques

R3 — Optimization-Techniques Deep-Web Research

Revision: 1 **Last modified:** 2026-05-20T00:00:00Z **Authority:** Cascaded from HelixCode root CONSTITUTION.md / CLAUDE.md. Research deliverable for the “ultra-fast” programme (3–5× faster than competitor AI CLI agents without breaking working features). Per §11.4.8 every technique cites a source URL. Per §11.4.74, where a technique implies a reusable component, a “Submodule-catalogue check” note flags whether the vasic-digital / HelixDevelopment owned-submodule catalogue should be consulted before building. Document type: research only — **no code changes, no commits implied by this file.**

Executive Summary

HelixCode’s “ultra-fast” target (3–5× over competitor AI CLI agents) is achievable but the leverage is **not evenly distributed**. The dominant wall-clock cost of an AI coding agent is **LLM round-trip latency**, not Go CPU time. Therefore the single highest-impact lever is **provider-side prompt caching** (Anthropic / OpenAI / Gemini): a warm codebase/system-prompt cache cuts time-to-first-token by up to ~85% and cost by ~90% on cache hits, with near-zero implementation risk. The second tier is **codebase-context speed** — incremental tree-sitter parsing + content-addressed caches turn a 22-minute cold index into a 45-second warm update. Only the third tier is classic Go performance engineering (PGO, allocation reduction, GC tuning, concurrency), which buys 2–30% on the CPU-bound slices that remain.

A realistic decomposition of the 3–5× goal:

Lever	Contribution to end-to-end speedup
Prompt caching + streaming-first + connection pooling	~2-3× on perceived latency (TTFT)
Incremental indexing + content-addressed context caches	~1.5-2× on context-build phases
Go PGO + allocation/GC/concurrency tuning	~1.05-1.3× on CPU-bound slices
Startup-time + build/test speed	developer-experience multiplier, not user-facing runtime

The techniques below are ordered by area; a consolidated impact-to-effort ranking closes the document.

Area 1 — Go Performance Engineering

1.1 Profile-guided optimization (PGO)

Modern Go (1.21+) auto-detects a `default.pgo` file in the main package directory and feeds a production CPU profile back into the compiler for better inlining/devirtualization. Collect via `net/http/pprof` (`curl .../debug/pprof/profile?seconds=30 > default.pgo`), commit the profile, rebuild — no flag needed (`-pgo=auto` is default). Go 1.22 benchmarks show **2-14%** gains; Uber reports it as a standard part of their efficiency pipeline. - Sources: <https://go.dev/doc/pgo>, <https://go.dev/blog/pgo>, <https://www.uber.com/blog/automating-efficiency-of-go-programs-with-pgo/>, <https://cloud.google.com/blog/products/application-development/using-profile-guided-optimization-for-your-go-apps> - **Impact:** Medium (2-14% CPU-bound). **Difficulty:** Low — one file + a profiling harness. - Submodule-catalogue check: the profile-collection harness is a reusable component; check the catalogue for an existing `helix_qa/perf-tooling` profiler runner before building one.

1.2 pprof-driven profiling workflow

`go tool pprof` on CPU + heap + mutex + block profiles is the mandatory first step — every later tuning decision (GC, allocation, contention) must be evidence-driven, never guessed (also a §11.4.6 no-guessing requirement). - Sources: <https://goperf.dev/01-common-patterns/gc/>, <https://dasroot.net/posts/2026/03/go-performance-optimization-profiling-techniques/> - **Impact:** Enabling (no direct speedup; unlocks everything else). **Difficulty:** Low.

1.3 Escape analysis & allocation reduction

go build -gcflags="-m" reveals heap escapes. Go 1.25 improved escape analysis + inlining, eliminating escapes after inlining. Keep hot-path objects stack-allocated; batch many small structs into one large slice to cut GC churn. - Sources: <https://gopherf.dev/01-common-patterns/gc/>, <https://reintech.io/blog/go-performance-optimization-guide-2026> - **Impact:** Medium-High on allocation-heavy paths (parsing, JSON, repo-map). **Difficulty:** Medium.

1.4 sync.Pool for short-lived objects

Reuse expensive transient allocations (buffers, parser scratch, request structs). A documented case: a server handling 10k concurrent connections cut CPU ~**30%** by replacing repeated map creation with a sync.Pool-backed cache. - Sources: <https://gopherf.dev/01-common-patterns/gc/>, <https://dasroot.net/posts/2026/03/go-performance-optimization-profiling-techniques/> - **Impact:** High on hot allocation paths. **Difficulty:** Medium (lifecycle correctness; never pool things crossing goroutine boundaries unsafely).

1.5 strings.Builder / bytes.Buffer over + concatenation

Repeated string + reallocates; strings.Builder grows once. Pre-size with Grow() when the final length is known. Relevant for repo-map rendering and prompt assembly. - Source: <https://gopherf.dev/01-common-patterns/gc/> - **Impact:** Medium on string-assembly hot paths. **Difficulty:** Low.

1.6 Struct layout & avoiding interface boxing

Order struct fields large→small to minimize padding; avoid storing scalars in interface{} (boxing forces heap allocation). Use concrete types in hot loops; reserve interfaces for boundaries. - Source: <https://reintech.io/blog/go-performance-optimization-guide-2026> - **Impact:** Low-Medium. **Difficulty:** Low-Medium.

1.7 GC tuning — GOGC, GOMEMLIMIT

GOGC (default 100) controls GC frequency; GOMEMLIMIT is a soft heap ceiling. For a container, set GOMEMLIMIT ~5–10% below the cgroup limit. Raising GOGC (or GOGC=off with a GOMEMLIMIT) trades memory for CPU on low-allocation workloads — Cloudflare reached **22×** in an extreme CPU-bound case at GOGC=11300; Uber's dynamic GC tuner saved ~70k cores. **Tune only after profiling confirms GC is the bottleneck** — the default is usually right. - Sources: <https://gopherf.dev/01-common-patterns/gc/>, <https://go.dev/doc/gc->

guide, <https://oneuptime.com/blog/post/2026-01-25-tune-garbage-collection/view> - **Impact:** Variable — Low default, High if GC-bound. **Difficulty:** Low (env vars) but risky without evidence.

Area 2 — Concurrency & Parallelism

2.1 errgroup with SetLimit() for bounded parallelism

golang.org/x/sync/errgroup gives error propagation + context cancellation + (via `SetLimit(n)`) a bounded worker pool in one primitive. Replace ad-hoc `WaitGroup`+channel fan-out for parallel file reads, parallel provider calls, parallel index shards. - Sources: <https://oneuptime.com/blog/post/2026-01-07-go-errgroup/view>, <https://encore.dev/blog/advanced-go-concurrency> - **Impact:** Medium-High (parallelizes I/O-bound work without unbounded goroutine blowup). **Difficulty:** Low.

2.2 Worker pools sized to the bottleneck

Size pools to `runtime.NumCPU()` for CPU-bound work, or to the downstream limit (DB pool, API concurrency) for I/O-bound work — never spawn one goroutine per unit of unbounded work. - Sources: <https://www.stanza.dev/courses/go-concurrency/patterns/go-concurrency-worker-pool>, <https://deepengineering.substack.com/p/deep-engineering-14-mihalis-tsoukalos> - **Impact:** Medium-High. **Difficulty:** Low-Medium. - Submodule-catalogue check: a generic bounded worker-pool / scheduler is a textbook reusable component — check `helix_code/internal/worker` and the owned-submodule catalogue before writing a new one.

2.3 Reducing mutex contention

Profile with the mutex/block profiler. Shrink critical sections; shard a hot map into N striped maps; prefer `sync.RWMutex` for read-heavy state; use `sync/atomic` for simple counters/flags; consider lock-free `atomic.Pointer` swap for copy-on-write config. - Sources: <https://encore.dev/blog/advanced-go-concurrency>, <https://goperf.dev/blog/2025/04/03/lazy-initialization-in-go-using-atomics/> - **Impact:** High when a hot lock is the bottleneck. **Difficulty:** Medium-High.

2.4 Channel vs mutex tradeoff

Channels for ownership transfer / pipelines; mutex/atomics for protecting in-place shared state. Channels add scheduling overhead — don't use them as a glorified lock around a counter. -

Source: <https://dev.to/ohugonnot/concurrency-in-go-part-2-channels-select-and-worker-pool-4g4o> - **Impact:** Low-Medium. **Difficulty:** Low.

Area 3 — Startup-Time Reduction

3.1 Lazy initialization (`sync.Once` / `atomic`)

Defer construction of any dependency taking >a few ms (LLM clients, DB pools, embedding models) until first use. For Go 1.24+, atomic-based lazy init avoids `sync.Once` overhead on the hot read path. - Sources: <https://goperf.dev/01-common-patterns/lazy-init/>, <https://goperf.dev/blog/2025/04/03/lazy-initialization-in-go-using-atomics/> - **Impact:** High on CLI cold-start (the user-facing “feels instant” metric). **Difficulty:** Low.

3.2 Non-blocking eager init (`inittrace` + `background init`)

For dependencies genuinely needed soon, kick off `sync.Once.Do` in a goroutine during `init()` so construction overlaps the rest of startup. Diagnose slow `init()` chains with `GODEBUG=inittrace=1`. - Source: <https://eblog.fly.dev/startfast.html> - **Impact:** Medium-High on cold-start. **Difficulty:** Low-Medium.

3.3 Embedded assets via `go:embed`

Bundle config templates / static assets with `go:embed` — no disk walk, no volume mounts. Parse embedded templates once at startup and fail fast. - Source: <https://oneuptime.com/blog/post/2026-02-01-go-embed-static-files/view> - **Impact:** Low-Medium. **Difficulty:** Low.

3.4 Fast config parsing

Parse config once, cache the struct, validate-and-fail-fast. Avoid re-reading YAML on every command; avoid heavyweight reflection-based decoders on the hot path. - Source: <https://eblog.fly.dev/startfast.html> - **Impact:** Low-Medium. **Difficulty:** Low.

Area 4 — LLM-Latency Reduction (highest leverage)

4.1 Provider-side prompt caching — Anthropic

Mark stable prefixes (system prompt, tool definitions, codebase context) with `cache_control: {"type": "ephemeral"}`. Cache reads cost **0.1×** input price (~90% savings); writes cost 1.25× (5-min TTL) or 2× (1-hour TTL, refreshed free on hit). Latency drops up to ~**85%** on long prompts (a 100k-token example: 11.5s → 2.4s). Up to 4 breakpoints per request; minimum 4,096 cacheable tokens for Opus 4.x. **Go pitfall:** Go randomizes JSON map key order — serialize tools/system blocks with deterministic key ordering or every request mishashes and never hits cache. - Sources: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>, <https://introl.com/blog/prompt-caching-infrastructure-llm-cost-latency-reduction-guide-2025>, <https://www.mager.co/blog/2026-04-29-claude-prompt-caching/> - **Impact:** Very High — single biggest user-facing win. **Difficulty:** Low-Medium (deterministic serialization is the only trap).

4.2 Cache pre-warming

At session start, fire a `max_tokens ~0` request to write the codebase/system-prompt cache so the first real user request is already a cache hit (no first-request penalty). - Source: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching> - **Impact:** High (removes the cold-start latency cliff). **Difficulty:** Low.

4.3 Streaming-first design

Stream tokens (SSE) so time-to-first-visible-token is the perceived latency, not time-to-full-completion. HelixCode already has `provider.GenerateStream` — ensure every surface (CLI, TUI, desktop) consumes streaming and renders incrementally. - Source: <https://www.tribe.ai/applied-ai/reducing-latency-and-cost-at-scale-llm-performance> - **Impact:** High on perceived latency. **Difficulty:** Low (mostly already present — verify, don't regress).

4.4 Small-model routing for cheap subtasks

Route trivial subtasks (intent classification, file-relevance ranking, commit-message drafting, ambiguity detection) to a fast small model (Haiku-class / local Ollama) and reserve the frontier model for actual code generation. Cascade architecture: cheap model first, escalate only on low confidence. - Sources: <https://www.tribe.ai/applied-ai/reducing-latency-and-cost-at-scale-llm->

performance, <https://introl.com/blog/speculative-decoding-llm-inference-speedup-guide-2025> - **Impact:** High on multi-step agent loops. **Difficulty:** Medium (routing policy + per-task model config — must source models from LLMsVerifier per CONST-036).

4.5 Speculative & parallel requests

Issue independent sub-requests in parallel (e.g., fetch context for N files concurrently); overlap action execution with ongoing reasoning. Provider-side speculative decoding gives 2–3× inference speedup where the provider/runtime supports it (relevant for self-hosted Ollama/Llama.cpp). - Sources: <https://introl.com/blog/speculative-decoding-llm-inference-speedup-guide-2025>, <https://arxiv.org/html/2511.20048v1> - **Impact:** Medium-High. **Difficulty:** Medium-High.

4.6 Request coalescing / batching

De-duplicate identical in-flight requests (singleflight); batch independent embedding / classification calls into one provider request. - Source: <https://www.tribe.ai/applied-ai/reducing-latency-and-cost-at-scale-llm-performance> - **Impact:** Medium. **Difficulty:** Low-Medium (golang.org/x/sync/singleflight).

4.7 Connection pooling / HTTP/2 keepalive

Go's default `http.Transport` allows only **2 idle connections per host** — fatal for a provider hammered with concurrent calls. Set `MaxIdleConns` and `MaxIdleConnsPerHost` to ~100, enable TCP `KeepAlive`: 30s, reuse one tuned `*http.Client` (never per-request clients), and use HTTP/2 (`Transport.Proto`) so many requests multiplex one connection. This alone eliminates per-request TLS-handshake latency. - Sources: <https://www.loginradius.com/blog/engineering/tune-the-go-http-client-for-high-performance>, <https://davidbacisin.com/writing/golang-http-connection-pools-1>, <https://goperf.dev/02-networking/efficient-net-use/> - **Impact:** High (removes handshake latency from every call). **Difficulty:** Low. - Submodule-catalogue check: a tuned HTTP-client factory is reusable across every provider — check the catalogue / `helix_code/internal/providers` for an existing shared transport.

Area 5 — Codebase-Context Speed

5.1 Incremental tree-sitter parsing

Tree-sitter re-parses only edited regions, sharing unchanged subtrees with the prior tree — parse-on-keystroke fast. Reported: autocomplete/error-check 60% faster (500ms → 200ms); a static analyzer did 1M LOC in <10s vs 22s with the prior parser. - Sources: <https://github.com/tree-sitter/tree-sitter>, <https://dasroot.net/posts/2026/02/incremental-parsing-tree-sitter-code-analysis/>, [https://en.wikipedia.org/wiki/Tree-sitter_\(parser_generator\)](https://en.wikipedia.org/wiki/Tree-sitter_(parser_generator)) - **Impact:** High on per-edit context refresh. **Difficulty:** Medium (HelixCode already vendors `smacker/go-tree-sitter` — wire the *incremental* edit API, don't full-reparse).

5.2 Content-addressed caching

Key context/index entries by a hash of source-file content. When a file changes its hash changes and dependent entries miss naturally — no explicit invalidation sweep. This is the correct invalidation primitive for repo-map, tags, and embedding caches. - Sources: <https://tianpan.co/blog/2026-04-20-cache-invalidation-ai-semantic-rag>, <https://sparkco.ai/blog/deep-dive-into-cache-invalidation-agents-in-2025> - **Impact:** High (correctness + speed). **Difficulty:** Low-Medium.

5.3 Incremental indexing with lineage tracking

On change recompute only the changed file's embedding/tags; on delete drop it; on unchanged reuse. Benchmark: 12k-file corpus full reindex 22 min → incremental 45 s. - Source: <https://tianpan.co/blog/2026-04-20-cache-invalidation-ai-semantic-rag> - **Impact:** Very High on warm context builds. **Difficulty:** Medium.

5.4 File-watch-driven invalidation (`fsnotify`)

Drive cache invalidation from filesystem events instead of polling/full rescan. HelixCode already depends on `fsnotify` — connect it to the content-addressed cache layer. - Sources: <https://sparkco.ai/blog/deep-dive-into-cache-invalidation-agents-in-2025>, <https://foojay.io/today/distributed-cache-invalidation-patterns/> - **Impact:** Medium-High. **Difficulty:** Low-Medium.

5.5 Repo-map compression / graph-ranked budgeting

Aider's model: build a file-dependency graph, rank with a PageRank-style algorithm, emit only the top-ranked definitions/signatures that fit the token budget; cache parsed tags (TAGS_CACHE), rendered snippets (tree_cache), and full maps (map_cache). This keeps the context dense and cheap to send. - Sources: <https://aider.chat/docs/repomap.html>, <https://deepwiki.com/Aider-AI/aider/4.1-repository-mapping-system> - **Impact:** High (smaller prompts → faster TTFT + cheaper). **Difficulty:** Medium-High. - Submodule-catalogue check: HelixCode has `helix_code/internal/repomap`; verify whether an owned submodule already implements graph-ranked repo mapping before extending.

5.6 Embedding caches

Cache embeddings keyed by content hash (content-addressed); never re-embed unchanged chunks. Combine with the multi-tier cache (Area 6). - Source: <https://tianpan.co/blog/2026-04-20-cache-invalidation-ai-semantic-rag> - **Impact:** High (embedding API calls are slow + costed). **Difficulty:** Low-Medium.

Area 6 — Caching Architecture

6.1 Multi-tier cache (memory → disk → Redis)

L1 in-process (~0.1 ms), L2 on-disk (persists across process restarts — critical for a CLI), L3 Redis (~1-5 ms, shared across instances/workers). Read L1→L2→L3, populate upward on hit. - Sources: <https://github.com/IBM/mcp-context-forge/issues/289>, <https://medium.com/@vinay.georgiatech/building-a-distributed-cache-system-a-deep-dive-into-caching-strategies-and-invalidation-patterns-44fb0c7a5376> - **Impact:** High. **Difficulty:** Medium. - Submodule-catalogue check: a multi-tier cache is a prime reusable component — check the owned-submodule catalogue and `helix_code/internal/redis` before building bespoke.

6.2 Cache-key design

Namespaced, versioned keys (`helix:repomap:v3:<contenthash>`); content-hash in the key for auto-invalidation; a version segment to invalidate a whole namespace by bumping the version. - Source:

<https://leapcell.io/blog/optimizing-database-performance-with-redis-cache-key-design-and-invalidation-strategies> - **Impact:** Medium (enabling — prevents stale-cache bugs). **Difficulty:** Low.

6.3 Invalidation strategy

Content-addressed keys (preferred — see 5.2), event-based pub/sub for distributed L3, Redis tag-sets to invalidate related entries together, write-through for must-be-fresh data. - Sources: <https://foojay.io/today/distributed-cache-invalidation-patterns/>, <https://redis.io/blog/guide-to-cache-optimization-strategies/> - **Impact:** Medium-High (correctness). **Difficulty:** Medium.

6.4 TTLs & eviction

TTLs sized to volatility (provider model lists short; parsed AST tags long); Redis LRU eviction (allkeys-lru/volatile-lru); align with CONST-038's $\leq 60s$ model-status freshness. - Source: <https://redis.io/blog/guide-to-cache-optimization-strategies/> - **Impact:** Medium. **Difficulty:** Low.

Area 7 — I/O Speed

7.1 Buffered I/O

Wrap file/network reads/writes in `bufio.Reader/bufio.Writer` to amortize syscalls — each syscall is $\sim 1\text{--}5\ \mu s$; hundreds of thousands of them dominate before any real disk work. - Source: <https://modulovalue.com/blog/syscall-overhead-tar-gz-io-performance/> - **Impact:** Medium-High on file-walk-heavy paths. **Difficulty:** Low.

7.2 Parallel file walking

Walk large repo trees with a bounded worker pool (`errgroup.SetLimit`) instead of a single `filepath.WalkDir` goroutine — directory stats and file reads parallelize well. - Sources: <https://goperf.dev/02-networking/efficient-net-use/>, <https://www.stanza.dev/courses/go-concurrency/patterns/go-concurrency-worker-pool> - **Impact:** Medium-High on cold index. **Difficulty:** Low-Medium.

7.3 mmap for large read-mostly files

Memory-map large files (large source files, index shards) for zero-copy reads — kernel page cache instead of explicit read syscalls + buffers. - Source: <https://medium.com/@alpesh.ccet/unleashing-io-performance-with-io-uring-a-deep-dive-54924e64791f> - **Impact:** Medium. **Difficulty:** Medium (lifecycle / unmap correctness; OS-specific).

7.4 io_uring for batched async I/O (Linux)

io_uring batches many I/O submissions into one syscall and supports zero-copy fixed buffers — amortizes syscall cost across many I/Os. Highest payoff for the index/embedding pipeline doing thousands of small reads. **Linux-only**; gate behind a build tag with a portable fallback. - Sources: <https://developers.mattermost.com/blog/hands-on-iouring-go/>, <https://iafisher.com/notes/2025/10/epoll-io-uring>, <https://arxiv.org/pdf/2512.04859> - **Impact:** High on the I/O-bound indexing path (Linux). **Difficulty:** High — only worth it if profiling proves syscalls dominate.

7.5 Minimize syscalls generally

Batch stat/open/read; read whole small files in one `os.ReadFile`; avoid per-line syscalls. Profile syscall count before optimizing. - Source: <https://moduloval.com/blog/syscall-overhead-tar-gz-io-performance/> - **Impact:** Medium. **Difficulty:** Low-Medium.

Area 8 — Build / Test Speed (developer-experience, not user runtime)

8.1 Go build cache

The build cache (`$GOCACHE`) makes unchanged packages near-free to rebuild — keep it warm in CI/dev containers; never wipe it between builds; PGO's first build rebuilds all packages but subsequent builds are cached. - Sources: <https://medium.com/@AlexanderObregon/go-build-cache-mechanics-6ada202c0502>, <https://www.codingexplorations.com/blog/mastering-gos-build-system-efficiency-through-caching-and-advanced-commands> - **Impact:** High on iteration speed. **Difficulty:** Low.

8.2 -p flag & build parallelism

-p controls how many packages/test binaries build/run in parallel (default = NumCPU). Tune GOMAXPROCS and -p to the host; in CPU-limited containers set GOMAXPROCS explicitly. - Sources: https://dev.to/emil_valeev/the-ultimate-guide-to-parallel-testing-in-go-p-parallel-and-tparallel-demystified-1c1o, <https://blog.howardjohn.info/posts/go-build-times/> - **Impact:** Medium-High. **Difficulty:** Low.

8.3 Test parallelism (t.Parallel() + -parallel)

-p = parallel test *packages*; -parallel = parallel test *functions within a package* (requires t.Parallel() in each test). Adding cores helps only if tests actually opt in. Note -count=1 (already in CLAUDE.md §3.4) defeats the test cache deliberately for honest runs. - Sources: <https://threedots.tech/post/go-test-parallelism/>, <https://bryce.is/writing/code/go-test-and-parallelism> - **Impact:** Medium-High on suite wall-time. **Difficulty:** Low-Medium (audit which tests are safely parallelizable — integration tests sharing infra may not be).

8.4 Compile-time reduction

Trim oversized packages, reduce deep dependency graphs, and reduce code-generation bloat; analyze with go build -debug-trace / build-time tooling. - Sources: <https://jsschools.com/golang/go-compilation-optimization-master-techniques-to-/>, <https://blog.howardjohn.info/posts/go-build-times/> - **Impact:** Medium. **Difficulty:** Medium-High.

Consolidated Ranking — Techniques by Impact-to-Effort

Tiers: **S** = do first (high impact, low effort), **A** = high value, **B** = solid, **C** = situational / evidence-gated.

Tier	Technique	Area	Impact	Effort
S	Anthropic-style prompt caching (deterministic JSON serialization)	4.1	Very High	Low-Med
S	Cache pre-warming at session start	4.2	High	Low
S	HTTP/2 + connection-pool tuning (MaxIdleConnsPerHost≈100)	4.7	High	Low

Tier	Technique	Area	Impact	Effort
S	Streaming-first on every surface (verify, don't regress)	4.3	High	Low
S	Content-addressed caching for repo-map / tags / embeddings	5.2	High	Low-Med
S	Lazy initialization of slow dependencies	3.1	High (cold-start)	Low
S	Go build cache kept warm + -p/GOMAXPROCS tuning	8.1/8.2	High (dev-exp)	Low
A	Incremental indexing with lineage tracking	5.3	Very High	Medium
A	Incremental tree-sitter parsing (use the edit API)	5.1	High	Medium
A	errgroup.SetLimit bounded parallelism (I/O fan-out, file walk)	2.1/7.2	Med-High	Low
A	Small-model routing for cheap subtasks (cascade)	4.4	High	Medium
A	sync.Pool on hot allocation paths (profile-confirmed)	1.4	High	Medium
A	Multi-tier cache (memory→disk→Redis)	6.1	High	Medium
A	PGO with a production default.pgo	1.1	Medium (2-14%)	Low
A	File-watch-driven invalidation (fsnotify → cache)	5.4	Med-High	Low-Med
A	Buffered I/O on file-walk paths	7.1	Med-High	Low
B	Repo-map graph-ranked compression (Aider model)	5.5	High	Med-High
B	Request coalescing / singleflight + embedding batching	4.6	Medium	Low-Med
B	Escape-analysis-driven allocation reduction	1.3	Med-High	Medium
B	Mutex-contention reduction (sharding / atomics / RWMutex)	2.3	High if locked	Med-High

Tier	Technique	Area	Impact	Effort
B	strings.Builder / pre-sized buffers	1.5	Medium	Low
B	Non-blocking eager init (inittrace)	3.2	Med-High	Low-Med
B	Test parallelism audit (t.Parallel() + -parallel)	8.3	Med-High (dev-exp)	Low-Med
B	go:embed assets + fast config parse	3.3/3.4	Low-Med	Low
B	Cache-key design + TTL/eviction policy	6.2/6.4	Medium	Low
C	GOGC/GOMEMLIMIT tuning (only if GC-bound by profile)	1.7	Variable	Low (risky)
C	Speculative / parallel sub-requests	4.5	Med-High	Med-High
C	mmap for large read-mostly files	7.3	Medium	Medium
C	io_uring batched async I/O (Linux, profile-gated)	7.4	High (I/O-bound)	High
C	Struct layout / interface-boxing avoidance	1.6	Low-Med	Low-Med
C	Compile-time reduction (dependency-graph trimming)	8.4	Medium	Med-High

Recommended sequencing

- Wave 1 (S-tier):** prompt caching + pre-warming + HTTP/2 pooling + streaming verification
 - content-addressed caching + lazy init. Low risk, $\sim 2\text{--}3\times$ perceived-latency win — gets most of the way to the $3\text{--}5\times$ target.
- Wave 2 (A-tier):** incremental indexing + incremental tree-sitter + bounded parallelism + small-model routing + multi-tier cache + PGO. Adds $\sim 1.5\text{--}2\times$ on context phases.
- Wave 3 (B/C-tier):** profile-driven allocation/GC/contention work + repo-map compression
 - I/O-layer tuning — only where pprof evidence proves the bottleneck (per §11.4.6).

Every wave must be guarded by the existing test/Challenge floor (CONST-048 four-layer coverage) so “ultra-fast” never trades away a working feature — speedups land with captured before/after runtime evidence per CONST-035 / Article XI §11.9.